

Wording for Individually Specializable Numeric Traits

Document #: WG21 P1841R2
Date: 2021-12-28
Audience: LWG
Reply to: Walter E. Brown <webrown.cpp@gmail.com>

Contents

1	Introduction	1	4	Bibliography	6
2	Proposed wording	2	5	Index of library names	7
3	Acknowledgments	6	6	Document history	7

Abstract

This paper combines/integrates/consolidates the wording from its predecessor papers [P0437R1] and [P1370R1].

[Why are numbers beautiful?] It's like asking why is Ludwig van Beethoven's Ninth Symphony beautiful. If you don't see why, someone can't tell you. I know numbers are beautiful. If they aren't beautiful, nothing is.

— PAUL ERDÖS

Two of the most famous Baghdadi scholars, the philosopher Al-Kindi and the mathematician Al-Khawarizmi, were certainly the most influential in transmitting Hindu numerals to the Muslim world. Both wrote books on the subject during al-Ma'mun's reign, and it was their work that was translated into Latin and transmitted to the West, thus introducing Europeans to the decimal system, which was known in the Middle Ages only as Arabic numerals. But it would be many centuries before it was widely accepted in Europe. One reason for this was sociological: decimal numbers were considered for a long time as symbols of the evil Muslim foe.

— JAMEEL SADIQ "JIM" AL-KHALILI

I'm writing a book. I've got the page numbers done.

— STEVEN WRIGHT

1 Introduction

This paper combines/integrates/consolidates the wording from its predecessor papers [P0437R1] and [P1370R1], each of which papers have been favorably reviewed by SG6 and LEWG. While those papers were originally approved for C++20, we opted to retard their progress due to LWG's huge backlog at the time. We now submit their combined proposed wording, adjusted per LEWG's review comments, for LWG review early in the C++23 cycle. Please see those earlier papers¹ for motivation, background information, examples, and other discussion of the present proposal.

Copyright © 2019–2022 by Walter E. Brown. All rights reserved.

¹There are no plans to revise either of those predecessor papers.

2 Proposed wording^{2,3}

2.1 After adjusting `yyyyymm` (below) so as to denote this proposal's month of adoption, insert the following line among the similar directives following `[version.syn]/2`:

```
#define __cpp_lib_numeric_traits yyyyymmL // also in <numbers>
```

2.2 Edit `[numerics.general]` as shown.

2 The following subclauses describe components for complex number types, random number generation, numeric (n -at-a-time) arrays, generalized numeric algorithms, traits for numeric types, and mathematical constants and functions for floating-point types, as summarized in Table `[tab:numerics.summary]`.

2.3 Edit `[support.limits.general]` as shown.

1 The headers `<limits>` (17.3.3), `<climits>` (17.3.6), and `<cfloat>` (17.3.7) supply implementation-dependent characteristics of ~~implementation-dependent~~ arithmetic types (6.8.1). In addition, the header `<numbers>` (`[num.traits]`) supplies components, collectively known as *numeric traits*, that not only provide implementation-dependent distinguished values and characteristics for arithmetic types, but that are (via specialization) individually extensible to provide analogous values and characteristics for program-defined numeric types.

2.4 Edit `[basic.fundamental]`, splitting the existing paragraph 12 where shown, updating the new paragraph 13's text as shown, and then incrementing the subsequent paragraph numbers in the subclause. [Note: The intent of these changes has been previously discussed with the C++17 Project Editor, who approved and contributed much of the following new wording. —end note]

12 The three distinct types `float`, `double`, and `long double` ... [Note: This document imposes no requirements on the accuracy of floating-point operations; see also 17.3. —end note]

13 Integral and floating-point types are collectively termed *arithmetic types*. ~~Specializations of the standard library template `std::numeric_limits` (17.3) shall specify the maximum and minimum values of each arithmetic type for an implementation.~~ [Note: Properties of the arithmetic types, such as their respective minimum and maximum representable value, can be queried using the facilities in the standard library headers `<limits>` (`[limits.syn]`), `<climits>` (`[climits.syn]`), `<cfloat>` (`[cfloat.syn]`), and `<numbers>` (`[num.traits.syn]`). —end note]

~~13~~14 A type `cv void` is ...

~~14~~15 A value of type `std::nullptr_t` is ...

~~15~~16 The types described in this subclause are ~~called~~termed *fundamental types*. [Note: Even if the implementation defines two or more fundamental types to have the same value representation, they are nevertheless different types. —end note]

²Proposed additions (entirely new subclauses are colored but not underlined) and ~~deletions~~ are based on [N4901]. Editorial instructions and drafting notes look like `this`.

³The $\LaTeX$ source of this wording is available to the Project Editor upon request.

2.5 Add the following new text to clause [support.limits], positioned at the discretion of the Project Editor. (Much of the wording specifying the individual traits is taken or adapted from the corresponding **numeric_limits** wording.) The Project Editor is authorized to delete the leading *// freestanding* comment from the synopsis in [num.traits.syn], below, should it be deemed necessary to do so for consistency with the remainder of the Working Draft.

17.3.x Numeric traits [num.traits]

17.3.x.1 General [num.traits.general]

1 Each numeric trait specified in this subclause [num.traits] is declared as a class template of the following form:

```
template <class T> struct TRAIT { };
```

2 Not all numeric traits are applicable to all numeric types. Each specialization of a numeric trait is either enabled or disabled, as described below.

3 An *enabled* specialization of a numeric trait is a trivial class type with no members other than trivial special member functions and a publicly accessible static data member named **value** that is usable in constant expressions. The type of the **value** member is described in the relevant subclause below.

4 A *disabled* specialization of a numeric trait has no members other than trivial special member functions. [Note: Every specialization that uses the primary template of a numeric trait is disabled. —end note]

5 For each numeric trait **TRAIT**<cv T>, the library provides partial specializations such that **TRAIT**<cv T> is enabled if and only if **TRAIT**<T> is enabled. When enabled, **TRAIT**<cv T>::value is equal to **TRAIT**<T>::value.

6 For each numeric trait, the library provides an enabled explicit specialization, if and as applicable, for each arithmetic type ([basic.fundamental]). In addition, a program may explicitly specialize a numeric trait for any cv-unqualified program-defined type ([numeric.requirements]) to which the numeric trait is applicable.

17.3.x.2 Header <numbers> synopsis [num.traits.syn]

```
// freestanding
namespace std::numbers {

    // [num.traits.val], numeric distinguished value traits
    template <class T> struct denorm_min           { };
    template <class T> struct epsilon              { };
    template <class T> struct finite_max           { };
    template <class T> struct finite_min           { };
    template <class T> struct infinity             { };
    template <class T> struct norm_min             { };
    template <class T> struct quiet_NaN            { };
    template <class T> struct reciprocal_overflow_threshold { };
    template <class T> struct round_error          { };
    template <class T> struct signaling_NaN        { };

    template <class T>
        inline constexpr auto denorm_min_v = denorm_min<T>::value;
    template <class T>
        inline constexpr auto epsilon_v = epsilon<T>::value;
    template <class T>
```

```

    inline constexpr auto finite_max_v = finite_max<T>::value;
template <class T>
    inline constexpr auto finite_min_v = finite_min<T>::value;
template <class T>
    inline constexpr auto infinity_v = infinity<T>::value;
template <class T>
    inline constexpr auto norm_min_v = norm_min<T>::value;
template <class T>
    inline constexpr auto quiet_NaN_v = quiet_NaN<T>::value;
template <class T>
    inline constexpr auto reciprocal_overflow_threshold_v
        = reciprocal_overflow_threshold<T>::value;
template <class T>
    inline constexpr auto round_error_v = round_error<T>::value;
template <class T>
    inline constexpr auto signaling_NaN_v = signaling_NaN<T>::value;

// [num.traits.char], numeric characteristics traits
template <class T> struct digits          { };
template <class T> struct digits10       { };
template <class T> struct max_digits10   { };
template <class T> struct max_exponent   { };
template <class T> struct max_exponent10 { };
template <class T> struct min_exponent   { };
template <class T> struct min_exponent10 { };
template <class T> struct radix          { };

template <class T>
    inline constexpr auto digits_v = digits<T>::value;
template <class T>
    inline constexpr auto digits10_v = digits10<T>::value;
template <class T>
    inline constexpr auto max_digits10_v = max_digits10<T>::value;
template <class T>
    inline constexpr auto max_exponent_v = max_exponent<T>::value;
template <class T>
    inline constexpr auto max_exponent10_v = max_exponent10<T>::value;
template <class T>
    inline constexpr auto min_exponent_v = min_exponent<T>::value;
template <class T>
    inline constexpr auto min_exponent10_v = min_exponent10<T>::value;
template <class T>
    inline constexpr auto radix_v = radix<T>::value;
}

```

17.3.x.4 Numeric distinguished value traits

[num.traits.val]

1 For the traits specified in this subclause, the **value** member of an enabled specialization has type **T**.

```
template <class T> struct denorm_min<T> { see below };
```

2 *Value*: **T**'s minimum positive denormalized value, if any; otherwise, the same value as **norm_min_v<T>**, if any.

```
template <class T> struct epsilon<T> { see below };
```

3 *Value*: $e - T(1)$, where e denotes T 's least value greater than 1, if any.

```
template <class T> struct finite_max<T> { see below };
```

4 *Value*: A finite value x , if any, such that T has no other finite value y such that $y > x$.

```
template <class T> struct finite_min<T> { see below };
```

5 *Value*: A finite value x , if any, such that T has no other finite value y such that $y < x$.

```
template <class T> struct infinity<T> { see below };
```

6 *Value*: T 's positive infinity, if any.

```
template <class T> struct norm_min<T> { see below };
```

7 *Value*: T 's minimum positive normalized value, if T supports subnormal numbers; otherwise, T 's minimum positive value.

```
template <class T> struct quiet_NaN<T> { see below };
```

8 *Value*: a quiet “Not a Number” value of T , if any.

```
template <class T> struct reciprocal_overflow_threshold<T> { see below };
```

9 *Value*: The smallest positive value x of type T such that $T(1)/x$ does not overflow.

```
template <class T> struct round_error<T> { see below };
```

10 *Value*: T 's maximum rounding error, if any.

```
template <class T> struct signaling_NaN<T> { see below };
```

11 *Value*: a signaling “Not a Number” value of type T , if any.

17.3.x.5 Numeric characteristics traits

[num.traits.char]

1 For the traits specified in this subclause, the **value** member of an enabled specialization has type **int**.

```
template <class T> struct digits<T> { see below };
```

2 *Value*: The number of **radix_v<T>** digits that can be represented without change. If **is_integer_v<T>** is **true**, this is the number of non-sign bits in the representation; If **is_floating_point_v<T>** is **true**, this is the number of **radix_v<T>** digits in the mantissa.

```
template <class T> struct digits10<T> { see below };
```

3 *Value*: The number of base 10 digits that can be represented without change.

```
template <class T> struct max_digits10<T> { see below };
```

4 *Value*: The number of base 10 digits required to ensure that type T values which differ are always differentiated.

```
template <class T> struct max_exponent<T> { see below };
```

5 *Value*: With $r = \text{radix_v}<\mathbf{T}>$, the largest positive integer i such that r^{i-1} is a representable finite value of type $\mathbf{T}$.

```
template <class T> struct max_exponent10<T> { see below };
```

6 *Value*: The largest positive integer i such that 10^i is in the range of representable finite type $\mathbf{T}$ values.

```
template <class T> struct min_exponent<T> { see below };
```

7 *Value*: With $r = \text{radix_v}<\mathbf{T}>$, the minimum negative integer i such that r^{i-1} is a representable, normalized (if applicable), finite value of type $\mathbf{T}$.

```
template <class T> struct min_exponent10<T> { see below };
```

8 *Value*: The minimum negative integer i such that 10^i is in the range of representable, normalized (if applicable), finite type $\mathbf{T}$ values.

```
template <class T> struct radix<T> { see below };
```

9 *Value*: The base of the representation. If `is_floating_point_v<T>` is `true`, this shall refer to the base or radix (often 2) of the exponent representation.

3 Acknowledgments

Many thanks to the readers of early drafts of this paper for their careful proofreading. Special thanks to Mark Hoemmen and Damien Lebrun-Grandie, authors of [P1370R1], for their support of this proposal and their contributions to it.

4 Bibliography

- [N4820] Richard Smith: “Working Draft, Standard for Programming Language C++.” ISO/IEC JTC1/SC22/WG21 document N4820 (pre-Cologne mailing), 2019-06-17. <https://wg21.link/n4820>.
- [N4861] Richard Smith: “Working Draft, Standard for Programming Language C++.” ISO/IEC JTC1/SC22/WG21 document N4861 (2020-04 mailing), 2020-04-01. <https://wg21.link/n4861>.
- [N4901] Thomas Köppe: “Working Draft, Standard for Programming Language C++.” ISO/IEC JTC1/SC22/WG21 document N4901 (2021-10 mailing), 2020-04-01. <https://wg21.link/n4901>.
- [P0437R1] Walter E. Brown: “Numeric Traits for the Standard Library.” ISO/IEC JTC1/SC22/WG21 document P0437R1 (pre-San Diego mailing), 2018-11-07. <https://wg21.link/p0437r1>.
- [P1370R1] Mark Hoemmen and Damien Lebrun-Grandie: “Generic numerical algorithm development with(out) `numeric_limits`.” ISO/IEC JTC1/SC22/WG21 document P1370R1 (post-San Diego mailing), 2019-03-10. <https://wg21.link/p1370r1>.
- [P2485R0] Jonathan Wakely: “Do not add `value_exists` and `value_or` to C++23.” ISO/IEC JTC1/SC22/WG21 document P2485R0 (2021-11 mailing), 2021-11-01. <https://wg21.link/p2485R0>.

5 Index of library names

<code>denorm_min</code> , 3 , 4	<code>max_exponent10_v</code> , 4
<code>denorm_min_v</code> , 3	<code>max_exponent_v</code> , 4
<code>digits</code> , 4 , 5	<code>min_exponent</code> , 4 , 6
<code>digits10</code> , 4 , 5	<code>min_exponent10</code> , 4 , 6
<code>digits10_v</code> , 4	<code>min_exponent10_v</code> , 4
<code>digits_v</code> , 4	<code>min_exponent_v</code> , 4
<code>epsilon</code> , 3 , 5	<code>norm_min</code> , 3 , 5
<code>epsilon_v</code> , 3	<code>norm_min_v</code> , 4
<code>finite_max</code> , 3 , 5	<code>quiet_NaN</code> , 3 , 5
<code>finite_max_v</code> , 4	<code>quiet_NaN_v</code> , 4
<code>finite_min</code> , 3 , 5	
<code>finite_min_v</code> , 4	
 	<code>radix</code> , 4 , 6
<code>infinity</code> , 3 , 5	<code>radix_v</code> , 4
<code>infinity_v</code> , 4	<code>reciprocal_overflow_threshold</code> , 3 , 5
 	<code>reciprocal_overflow_threshold_v</code> , 4
<code>max_digits10</code> , 4 , 5	<code>round_error</code> , 3 , 5
<code>max_digits10_v</code> , 4	<code>round_error_v</code> , 4
<code>max_exponent</code> , 4 , 6	
<code>max_exponent10</code> , 4 , 6	
	<code>signaling_NaN</code> , 3 , 5
	<code>signaling_NaN_v</code> , 4

6 Document history

Rev.	Date	Changes
0	2019-08-02	• Based wording on [N4820] . • Published as [P1841R0], post-Cologne mailing.
1	2020-05-03	• Rebased on [N4861] . • Applied several wording improvements. • Published as [P1841R1], 2020-05 mailing.
2	2021-12-28	• Rebased on [N4901] . • Applied some typographical improvements. • Revised Proposed Wording per LWG reviews of R1. • Adopted specific wording suggestions privately submitted by the LWG Chair. • Excised Proposed Wording for the numeric utility traits per LEWG-approved [P2485R0] . • Published as [P1841R2], 2022-01 mailing.
